```
»       def ensure_cart_isnt_empty
»         if @cart.line_items.empty?
»           redirect_to store_index_url, notice: 'Your cart is empty'
»         end
»       end
  end
```

If nothing is in the cart, we redirect the user back to the storefront, provide a notice of what we did, and return immediately. This prevents people from navigating directly to the checkout option and creating empty orders. Note that we tucked this handling of an exception case into a `before_action` method. This enables the main line processing logic to remain clean.

And we add a test for `requires item in cart` and modify the existing test for `should get new` to ensure that the cart contains an item:

rails7/depot_o/test/controllers/orders_controller_test.rb

```
»   test "requires item in cart" do
»     get new_order_url
»     assert_redirected_to store_index_path
»     assert_equal 'Your cart is empty', flash[:notice]
»   end

    test "should get new" do
»     post line_items_url, params: { product_id: products(:ruby).id }
»
      get new_order_url
      assert_response :success
    end
```

Now we want the `new` action to present users with a form, prompting them to enter the information in the `orders` table: the user's name, address, email address, and payment type. This means we'll need to display a Rails template containing a form. The input fields on this form will have to link to the corresponding attributes in a Rails model object, so we need to create an empty model object in the `new` action to give these fields something to work with.

As always with HTML forms, the trick is populating any initial values into the form fields and then extracting those values out into our application when the